

Implementação do Simply Typed Lambda Calculus em Rust

Nícolas Rosenthal Dal Corso

2026

Contents

1	Introdução	1
2	Termos	2
3	variáveis;	3
4	o valor booleano true;	3
5	o valor booleano false;	3
6	expressões condicionais;	3
7	abstrações lambda;	3
8	aplicações de funções.	3
8.1	Sintaxe concreta	4
8.2	Variáveis	4
8.3	Valores booleanos	4
8.4	Expressões condicionais	5
8.5	Abstrações lambda	5
8.6	Aplicações	6
8.7	Correspondência entre a sintaxe e a implementação	7

1 Introdução

Este relatório apresenta a implementação, em Rust, do **Simply Typed Lambda Calculus** (STLC), seguindo a formalização apresentada por Ben-

jamin C. Pierce em **Types and Programming Languages** (TAPL).

O cálculo lambda simplesmente tipado constitui uma extensão do cálculo lambda não tipado na qual cada parâmetro de uma função recebe um tipo explícito. Além das abstrações e aplicações características do cálculo lambda, a linguagem considerada nesta implementação possui valores booleanos e expressões condicionais. Dessa forma, a implementação contempla os elementos fundamentais necessários para estudar a relação entre sintaxe, tipagem e avaliação de uma linguagem funcional tipada.

A implementação foi desenvolvida em Rust, utilizando tipos algébricos representados por *enums* e estruturas recursivas por meio de `Box<T>`. Essa representação permite modelar diretamente a estrutura abstrata dos termos da linguagem e, ao mesmo tempo, aproveitar o sistema de tipos da própria linguagem Rust para garantir propriedades estruturais da implementação.

A linguagem é organizada em torno de três componentes principais: os **termos**, que definem a sintaxe das expressões; os **tipos**, que descrevem as categorias sintáticas às quais os termos pertencem; e as regras de avaliação e tipagem, que determinam quais expressões são válidas e como elas são reduzidas.

Neste relatório, a implementação é apresentada a partir de sua correspondência com a definição formal da linguagem. Inicialmente são descritos os termos e sua representação em Rust; em seguida, podem ser apresentados os tipos, as regras de tipagem e a avaliação operacional da linguagem.

2 Termos

Os termos da linguagem correspondem às expressões que podem ser construídas pelo STLC. A implementação considera seis formas sintáticas fundamentais:

- 3 variáveis;
- 4 o valor booleano `true`;
- 5 o valor booleano `false`;
- 6 expressões condicionais;
- 7 abstrações `lambda`;
- 8 aplicações de funções.

Em Rust, essas formas são representadas pelo tipo algébrico `Term`:

```
use std::collections::BTreeSet;
use std::fmt;

use super::types::Type;

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Term {
    /// `x`
    Var(String),
    /// `true`
    True,
    /// `false`
    False,
    /// `if c then t else e`
    If { condition: Box<Term>, then_branch: Box<Term>, else_branch: Box<Term> },
    /// `x:T. body`
    Lambda { param: String, ty: Type, body: Box<Term> },
    /// `f x`
    App { func: Box<Term>, arg: Box<Term> },
}
```

A enumeração `Term` constitui uma representação direta da sintaxe abstrata da linguagem. Cada variante corresponde a uma construção sintática específica.

8.1 Sintaxe concreta

A sintaxe dos termos pode ser descrita pela seguinte gramática BNF:

$$\begin{array}{c} t ::= x \\ | \text{ true} \\ [\text{ false} \quad | \quad \text{if } t \text{ then } t \text{ else } t \quad] \\ | \lambda x. t \\ t \, t \end{array}$$

Em notação BNF, a mesma gramática pode ser apresentada como:

$$[\, t \, ::= \, x \, | \, \text{true} \, | \, \text{false} \, | \, \text{if } t \text{ then } t \text{ else } t \, | \, \lambda x. t \, | \, t \, t \,]$$

onde t representa um termo, x representa uma variável e T representa um tipo.

A gramática estabelece apenas quais expressões pertencem à sintaxe da linguagem. A validade tipada dessas expressões é determinada posteriormente pelas regras de tipagem.

8.2 Variáveis

Uma variável é representada pela produção

$$[\, t ::= x. \,]$$

Na implementação, essa construção corresponde à variante:

`Var(String)`

O identificador da variável é armazenado como uma `String`. Assim, por exemplo, a expressão matemática

$$[\, x \,]$$

pode ser representada por:

`Term::Var("x".to_string())`

A utilização de `String`, em vez de índices inteiros ou de uma estrutura separada para identificadores, mantém a representação próxima da notação utilizada na apresentação formal do cálculo.

8.3 Valores booleanos

A linguagem possui dois valores booleanos:

$$[\, \text{true} \quad \text{false}. \,]$$

Eles são representados diretamente pelas variantes:

```
True,  
False,
```

correspondendo respectivamente a:

```
[ true : Bool ]  
e  
[ false : Bool. ]
```

8.4 Expressões condicionais

Uma expressão condicional possui a forma

```
[ if\ t_1\ then\ t_2\ else\ t_3. ]
```

Na implementação, ela é representada por:

```
If {  
  condition: Box<Term>,  
  then_branch: Box<Term>,  
  else_branch: Box<Term>,  
}
```

Os três campos são termos porque tanto a condição quanto os dois ramos podem conter expressões arbitrariamente complexas.

Por exemplo,

```
[ if\ true\ then\ x\ else\ y ]
```

pode ser representado por:

```
Term::If {  
  condition: Box::new(Term::True),  
  then_branch: Box::new(Term::Var("x".to_string())),  
  else_branch: Box::new(Term::Var("y".to_string())),  
}
```

A utilização de `Box<Term>` é necessária porque `Term` é um tipo recursivo: um termo condicional contém outros termos.

8.5 Abstrações lambda

A abstração lambda constitui a construção utilizada para representar funções. No STLC, diferentemente do cálculo lambda não tipado, o parâmetro da função possui um tipo explícito:

```
[ λ x.\ t. ]
```

A representação em Rust é:

```

Lambda {
  param: String,
  ty: Type,
  body: Box<Term>,
}

```

O campo `param` armazena o nome do parâmetro, enquanto `ty` armazena seu tipo. O corpo da função é novamente um `Term` e, portanto, é armazenado indiretamente através de `Box<Term>`.

Por exemplo, a função identidade sobre booleanos,
 $[\lambda x:\text{Bool}.x,]$
 pode ser representada por:

```

Term::Lambda {
  param: "x".to_string(),
  ty: Type::Bool,
  body: Box::new(Term::Var("x".to_string())),
}

```

A presença do tipo no próprio termo permite representar diretamente a sintaxe anotada do STLC.

8.6 Aplicações

A aplicação de uma função a um argumento possui a forma

$[t_1 \backslash t_2.]$

Ela é representada pela variante:

```

App {
  func: Box<Term>,
  arg: Box<Term>,
}

```

Por exemplo, a aplicação da função identidade booleana ao valor `true`,
 $[(\lambda x:\text{Bool}.x) \backslash \text{true},]$
 pode ser representada por:

```

Term::App {
  func: Box::new(Term::Lambda {
    param: "x".to_string(),
    ty: Type::Bool,

```

```

body: Box::new(Term::Var("x".to_string())),
}),
arg: Box::new(Term::True),
}

```

Assim como ocorre com as demais construções recursivas, `Box<Term>` permite que uma aplicação contenha termos arbitrariamente complexos tanto na posição de função quanto na posição de argumento.

8.7 Correspondência entre a sintaxe e a implementação

A correspondência entre a gramática e a representação em Rust pode ser resumida da seguinte forma:

Construção sintática	Representação Rust	height
<code>x Term::Var(String)</code>	<code>true Term::True</code>	<code>false Term::False</code>
<code>if t_1 then t_2 else t_3</code>	<code>Term :: If $\lambda x. t$ Term</code>	

Essa correspondência é intencionalmente próxima da definição formal. A estrutura recursiva de `Term` reproduz a estrutura da árvore sintática abstrata: cada termo composto contém subtermos representados por outros valores de `Term`.

A derivação dos tipos desses termos, bem como as regras de avaliação, pode ser definida independentemente da representação concreta utilizada pelo programa. Isso permite separar a definição formal da linguagem de sua implementação e utilizar a enumeração `Term` como uma representação computacional da sintaxe abstrata.